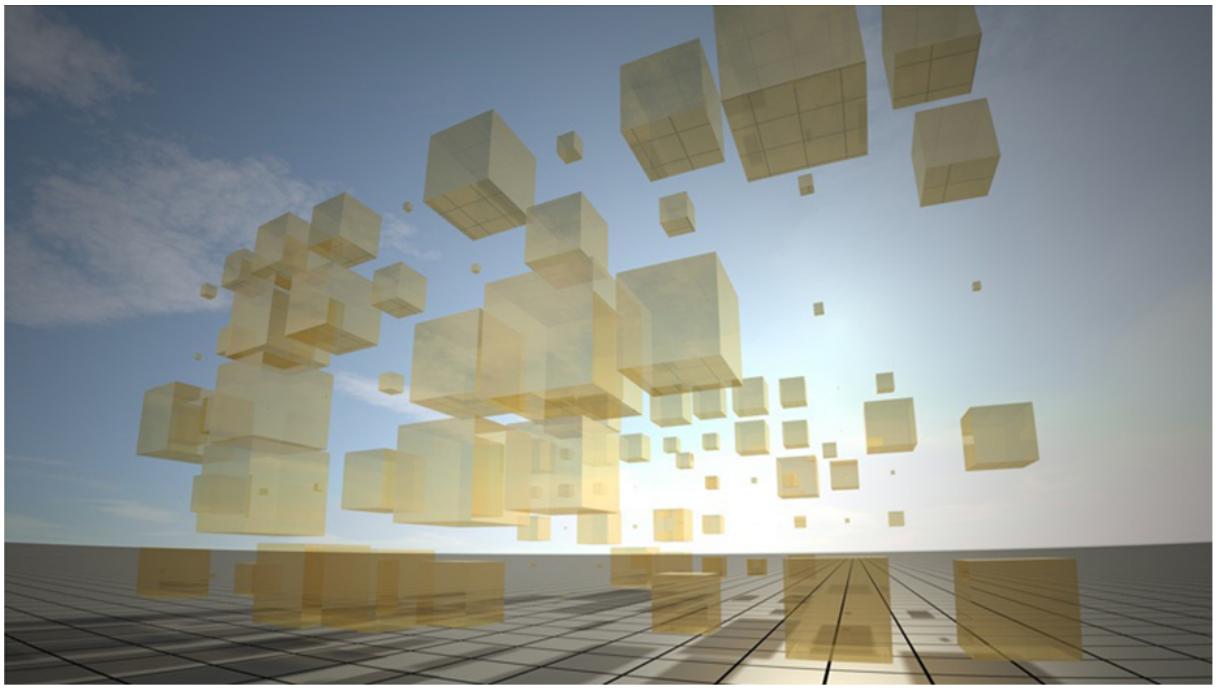
Hochleistungsdatenhaltungssysteme



Dr. Stephan Hofmann

TH Ingolstadt

Teil 2.1: Umgang mit Tabellen

Überblick

Die folgende Übung gibt eine Einführung in Konzepte und Werkzeuge zur Verwendung von Tabellen innerhalb der SAP Web IDE.

Einsatzszenario:

Die SAP Web IDE ist eine auf XSA basierende Webanwendung innerhalb der SAP HANA Plattform und ist ein modernes Entwicklungswerkzeug SAP HANA. Somit ist es wichtig einiges über die Einsatzmöglichkeiten der SAP Web IDE zu erfahren und wie man als Administrator und Entwickler damit arbeitet.

Zielsetzung:

Nach dem Abschluss der Übung sollten folgende Inhalte bekannt und verstanden sein:

- Anlegen von Tabellen in SAP HANA
- Einfügen von Inhalten in existierende Tabellen
- Definieren von einfachen Abfragen mit Hilfe von SQL
- Verknüpfung von Tabellen mit Hilfe von Join-Anfragen
- Persistieren von SQL-Anfragen

Inhalt

Arbeiten mit Tabellen in SAP WEBIDE 4
Hinweis: 4
Erzeugen einer Tabelle in SAP Web IDE: 4
Erzeugen einer Tabelle mit SQL 4
Einfügen von Daten in die Tabelle: 6
Einfügen per SQL 6
Einfügen per Datei-Import 9
Hinzufügen einer neuen Spalte zur bestehenden Tabelle: 12
Einsatz von Join-Operationen mit Hilfe SQL in SAP HANA 14
Anlegen der notwendigen Tabellen 14
Definition der verfügbaren Join-Operationen 16
Inner Join 17
SQL Outer Joins 18
Left Outer Join - 18
Right Outer Join - 19
Full Outer Join - 20
Persistieren von Anfragen 21
Import der Anweisungen aus einer lokalen Datei 21
Export der Anweisungen in eine lokale Datei 21
Selektives Ausführen einzelner Anfragen 22

Arbeiten mit Tabellen in SAP WEBIDE

Hinweis:

Bitte beachten Sie, da wir für die Operationen alle auf dem gleichen Datenbankschema arbeiten, müssen die Namen der Artefakte anhand folgender Regel angepasst werden.

Zur eindeutigen Unterscheidung aller Artefakte: Tabellen, Views wird die jeweilige <Nummer> des Benutzers vor den Namen gesetzt.

Benutzer: D02_LEARN_xxx → D02_LEARN_001

Tabellen-Name: xxx_Test_Table1. → D001_Test_Table1

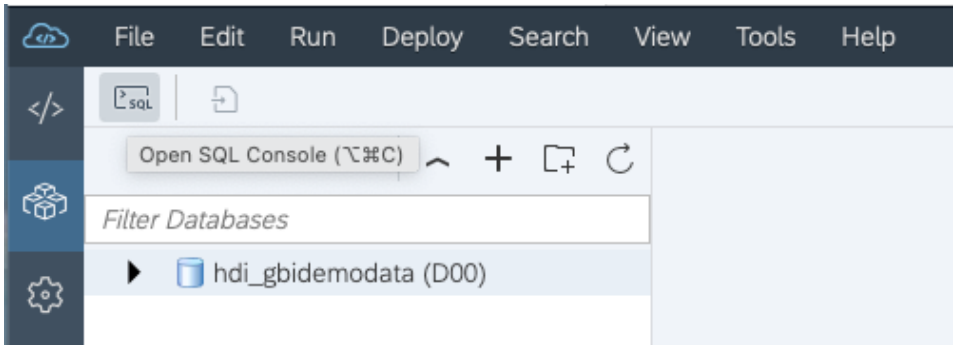
Erzeugen einer Tabelle in SAP Web IDE:

Innerhalb der SAP Web IDE können Tabellen per SQL angelegt werden.

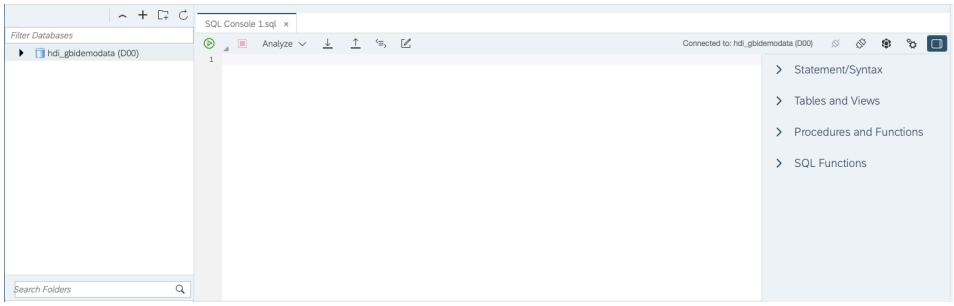
Erzeugen einer Tabelle mit SQL

Tabellen können mit Hilfe von Standard-SQL Befehlen innerhalb der SQL-Konsole erzeugt werden.

Zum Öffnen einer SQL-Konsole, wählt man den relevanten HDI-Container aus und klickt dann entweder auf den SQL-Button am oberen Rand der Systemübersicht oder man benutzt das Kontextmenü und wählt „Open SQL Console“ aus.



Daraufhin öffnet sich ein SQL-Editor Fenster.



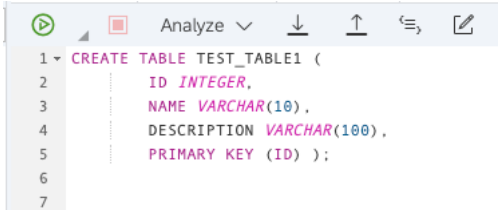
Das nachfolgende SQL-Statement erzeugt eine Tabelle (TEST_TABLE_1) innerhalb der HANA-DB mit drei Spalten.

```
CREATE TABLE xxx_TEST_TABLE1(
        ID INTEGER,
        NAME VARCHAR(10),
        DESCRIPTION VARCHAR(100),
        PRIMARY KEY (
                ID
        )
)
```

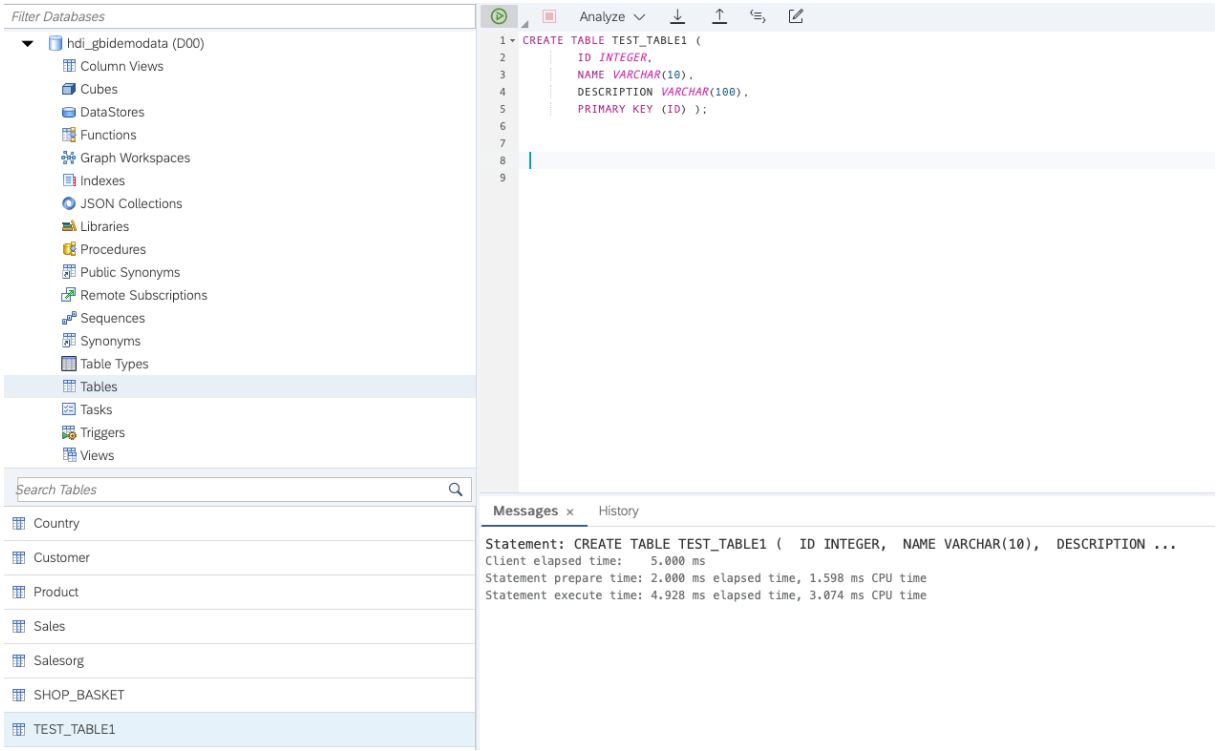
Die SQL-Konsole enthält zur besseren Lesbarkeit der Anfragen und Minimierung von Fehlern ein eingebautes Syntax-Highlighting.

Zum Ausführen einer Anfrage drückt man Ausführen Button

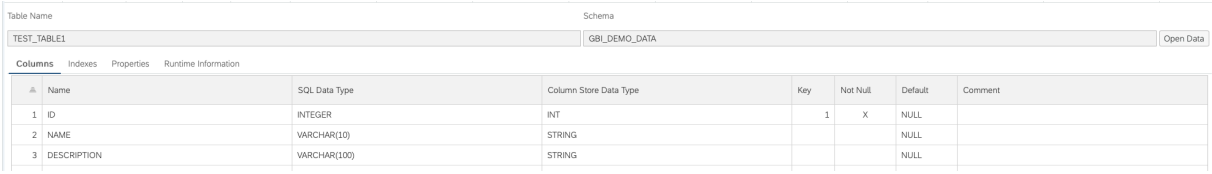
.



Nach dem Ausführen der Anweisung erscheint die entsprechende Meldung im Detailfenster und im Navigatorbaum erscheint der Name der erzeugten Tabelle.



Um sich die Definition der Tabelle ansehen zu können, klickt man entweder doppelt auf den Tabellennamen oder öffnet die Tabellendefinition über das Kontextmenü. Im Editorbereich wird jetzt die Tabellendefinition dargestellt.

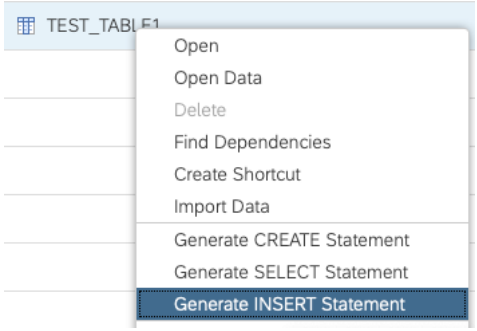


Einfügen von Daten in die Tabelle:

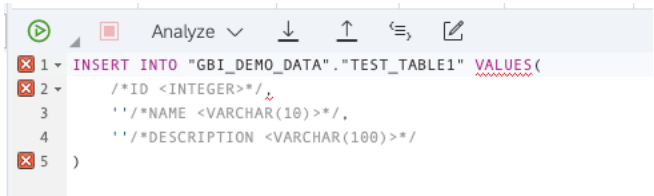
Einfügen per SQL

Daten können ebenfalls per Standard-SQL in die Tabellen eingefügt werden. Die Daten können danach in der „Data Preview“ angezeigt werden.

Um für eine bestehende Tabelle ein entsprechendes Insert-Template zu generieren nutzt man das Kontextmenü.



Das vorbereitete Statement wird angezeigt und kann angepasst oder ersetzt werden.




```
INSERT INTO xxx_TEST_TABLE1 VALUES(
        /*ID <INTEGER>*/,
       ''/*NAME <VARCHAR(10)>*/,
       ''/*DESCRIPTION <VARCHAR(100)>*/
)
```

Zunächst sind alle Anweisungen, die auf die einzelnen Spalten der Tabelle zeigen auskommentiert. Die Anweisung ist so nicht ausführbar, da noch keine expliziten Werte angegeben sind, der integrierte Code-Check zeigt entsprechende Fehlermeldungen an.

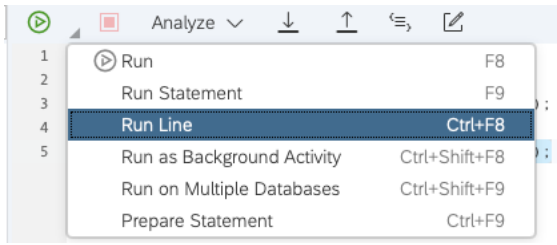
Wir fügen exemplarisch die folgenden Werte in die Anweisungen ein:

```
INSERT INTO xxx_TEST_TABLE1 VALUES(
       1 /*ID <INTEGER>*/,
       'A'/*NAME <VARCHAR(10)>*/,
       'A12345'/*DESCRIPTION <VARCHAR(100)>*/
)
```

und führen die Anweisung aus.

```
INSERT INTO "GBI_DEMO_DATA"."TEST_TABLE1" VALUES(
    1 /*ID <INTEGER>*/,
    'A'/*NAME <VARCHAR(10)>*/,
    'A12345'/*DESCRIPTION <VARCHAR(100)>*/
)
```

Danach führen wir noch die folgenden Anweisungen via



aus.

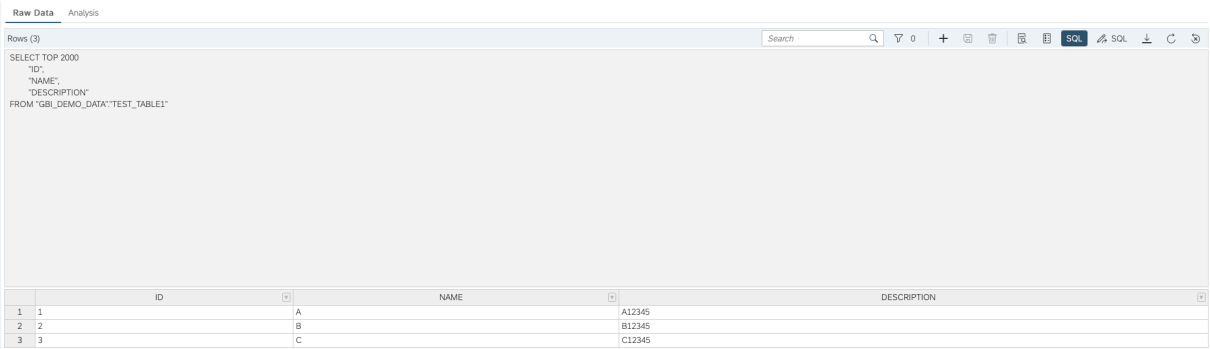
```
INSERT INTO D001_TEST_TABLE1 VALUES (6, 'G', 'C12345');

INSERT INTO D001_TEST_TABLE1 VALUES (7, 'H', 'B12345');
```

```
Messages × History

Statement: INSERT INTO TEST_TABLE1 VALUES (3, 'C', 'C12345')
Rows affected:          1
Client elapsed time:    2.000 ms
Statement prepare time: 4.237 ms elapsed time, 3.721 ms CPU time
Statement execute time: 0.8880 ms elapsed time, 0.6840 ms CPU time
```

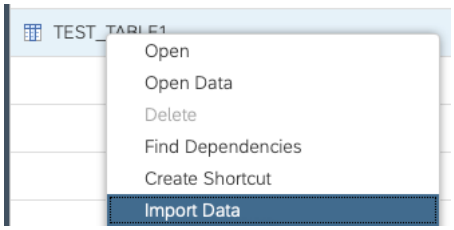
Über das Kontextmenü auf der betreffenden Tabelle kann man über "Open Data", den Inhalt der Tabelle angezeigt bekommen. Hierzu wird ein Select-Statement automatisch generiert und ausgeführt.

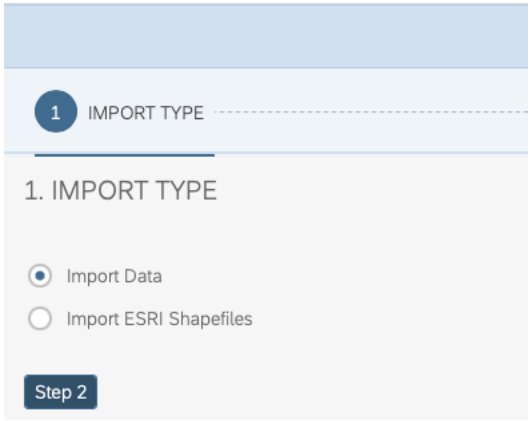


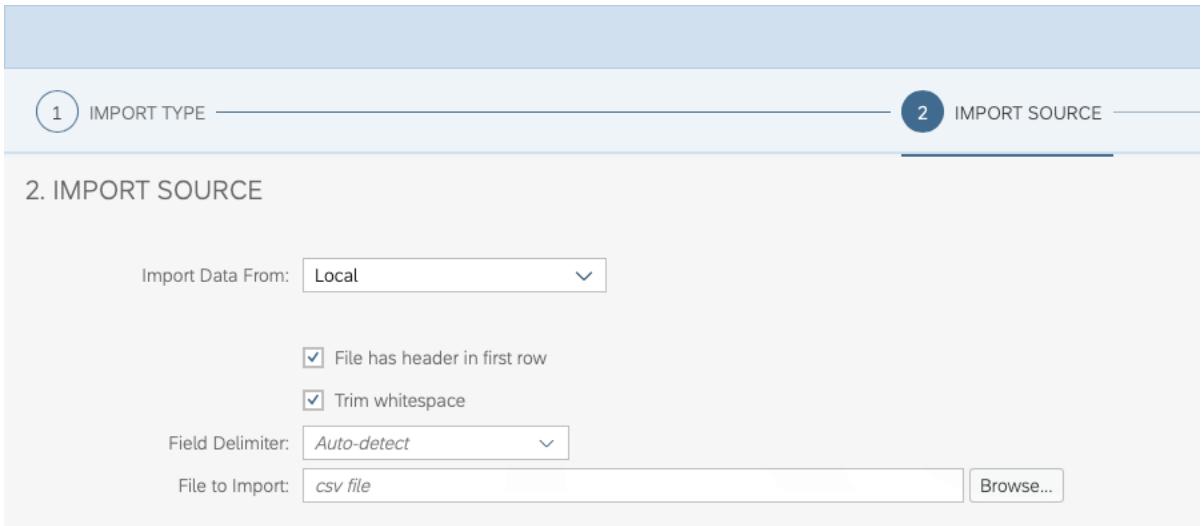
Einfügen per Datei-Import

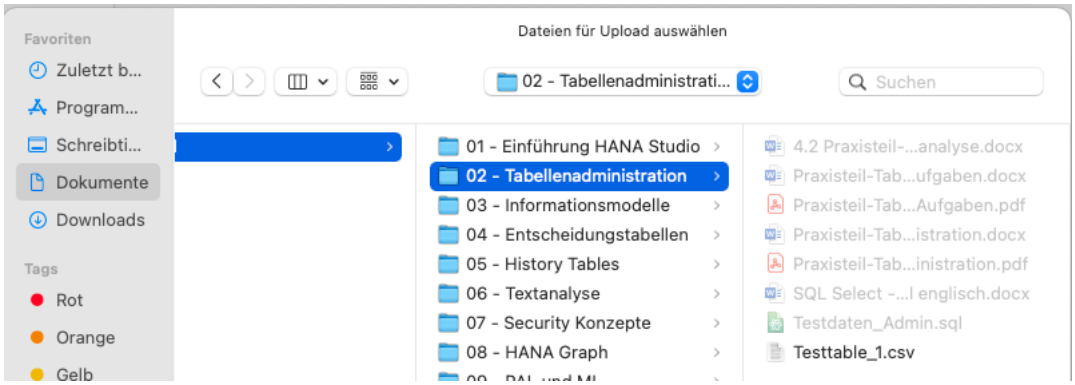
Daten können jedoch auch aus einer passenden CSV-Datei importiert werden.

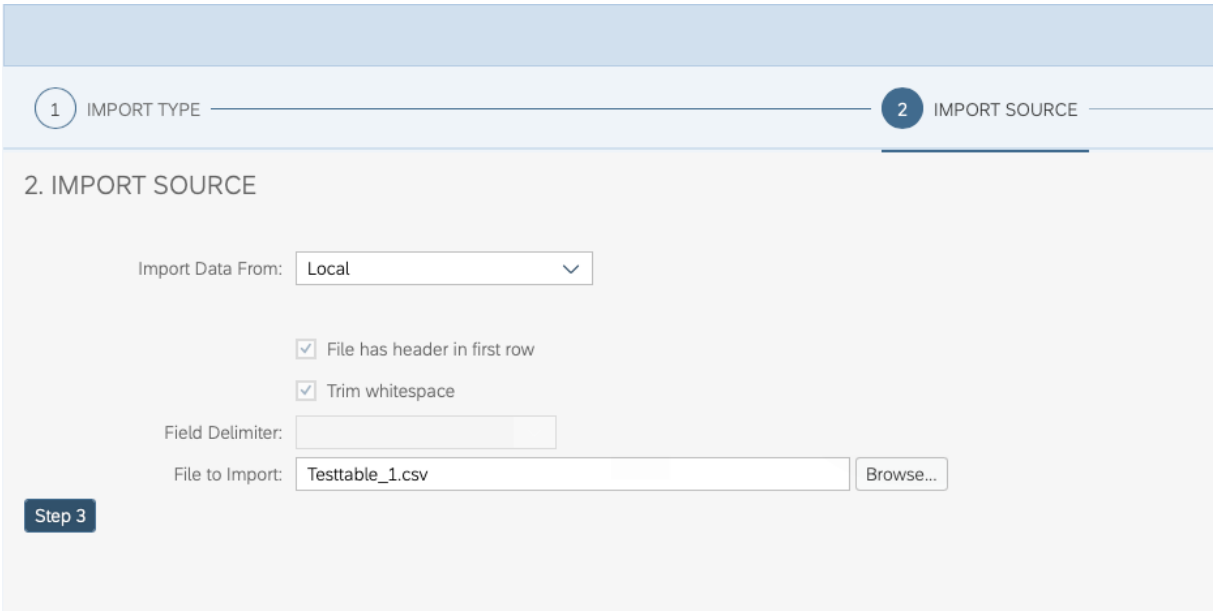
Hierzu kann die entsprechende Funktion über das Kontext-Menü genutzt werden.

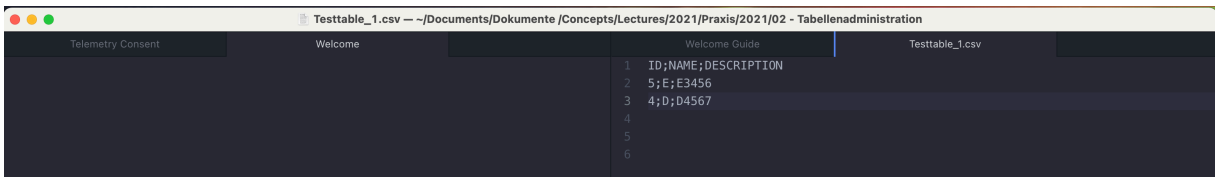


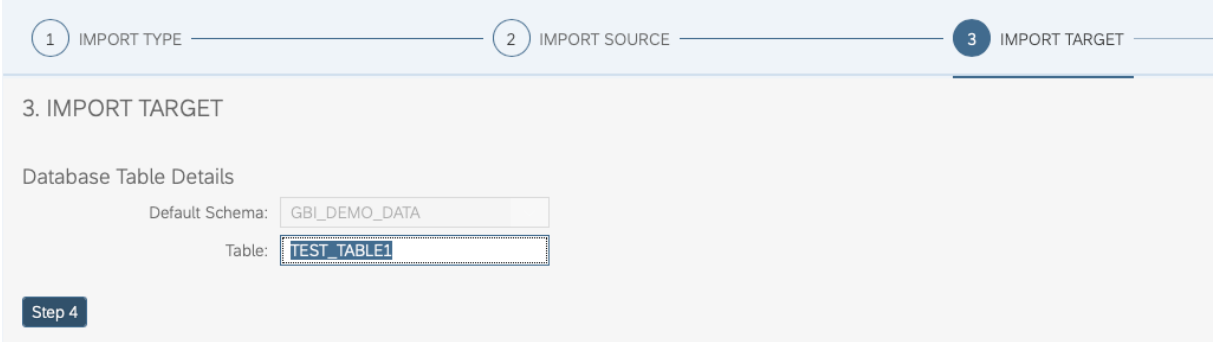





Favoriten
Zuletzt b...
Program...
Schreibti...
Dokumente
Downloads
Tags
Rot
Orange
Gelb
Dateien für Upload auswählen
02 - Tabellenadministrati...
Suchen
01 - Einführung HANA Studio
02 - Tabellenadministration
03 - Informationsmodelle
04 - Entscheidungstabellen
05 - History Tables
06 - Textanalyse
07 - Security Konzepte
08 - HANA Graph
4.2 Praxisteil-...analyse.docx
Praxisteil-Tab...ufgaben.docx
Praxisteil-Tab...Aufgaben.pdf
Praxisteil-Tab...istration.docx
Praxisteil-Tab...inistration.pdf
SQL Select -...l englisch.docx
Testdaten_Admin.sql
Testtable_1.csv


1 IMPORT TYPE
2 IMPORT SOURCE
2. IMPORT SOURCE
Import Data From: Local
File has header in first row
Trim whitespace
Field Delimiter:
File to Import: Testtable_1.csv
Browse...
Step 3


Testtable_1.csv — ~/Documents/Dokumente /Concepts/Lectures/2021/Praxis/2021/02 - Tabellenadministration
Telemetry Consent
Welcome
Welcome Guide
Testtable_1.csv
ID;NAME;DESCRIPTION
5;E;E3456
4;D;D4567


1 IMPORT TYPE
2 IMPORT SOURCE
3 IMPORT TARGET
3. IMPORT TARGET
Database Table Details
Default Schema: GBI_DEMO_DATA
Table: TEST_TABLE1
Step 4

Import Data

1 IMPORT TYPE — 2 IMPORT SOURCE — 3 IMPORT TARGET — 4 TABLE MAPPING — 5 ERROR HANDLING

4. TABLE MAPPING

Adjust table properties, map source, and database columns

HANA Database Column Properties:

Table Type: Column Store

Source Column Name	Use Default Value	Database Column Name	Data Type	Length	Scale	Key	Not Null
ID	☐	ID	INTEGER	10	0	☑	☑
NAME	☐	NAME	VARCHAR	10		☐	☐
DESCRIPTION	☐	DESCRIPTION	VARCHAR	100		☐	☐

Step 5

Import Data

1 IMPORT TYPE — 2 IMPORT SOURCE — 3 IMPORT TARGET — 4 TABLE MAPPING — 5 ERROR HANDLING

5. ERROR HANDLING

In case of an error, how do you want the system to handle it?

- (•) Save all successful rows and list the errors (if any)
- ( ) Do not save any rows if there is an error
- ( ) Show me the error rows and let me decide

Review

Import Summary

Import Into Database Cancel

IMPORT SOURCE

Edit

File Name:	Testtable_1.csv
File Has Header:	Yes
Table Exists:	Yes
Schema Name:	GBI_DEMO_DATA
Database Table Name:	TEST_TABLE1
Table Type:	Column Store

TABLE MAPPING

Edit

HANA database column properties:

Source Column Name	Use Default Value	Database Column Name	Data Type	Length	Scale	Key	Not Null
ID	☐	ID	INTEGER	10	0	☑	☑
NAME	☐	NAME	VARCHAR	10		☐	☐
DESCRIPTION	☐	DESCRIPTION	VARCHAR	100		☐	☐

ERROR HANDLING

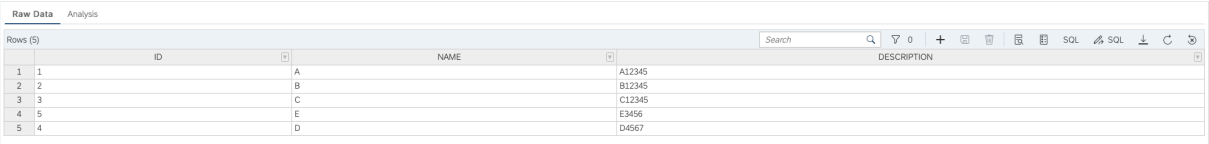
Edit

Error Handling:	Save all successful rows and list the errors (if any)

Data Preview:

Data preview based on customizations done in table mapping

ID	NAME	DESCRIPTION
5	E	E3456
4	D	D4567



	ID	NAME	DESCRIPTION
1	1	A	A12345
2	2	B	B12345
3	3	C	C12345
4	5	E	E3456
5	4	D	D4567

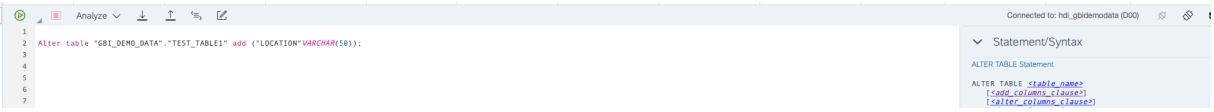
Hinzufügen einer neuen Spalte zur bestehenden Tabelle:

Mit Hilfe der folgenden Anweisung können auch nachträglich noch weitere Spalten hinzugefügt werden:

Alter table "<schema>"."<tabellen name> " add ("spalte " <datentyp>);

Ein Beispiel:

ALTER TABLE "GBI_DEMO_DATA"."xxx_TEST_TABLE1" add ("LOCATION" VARCHAR(50));



Das Ergebnis kann wieder über das Kontextmenü angesehen werden.

Table Name: TEST_TABLE1 | Schema: GBI_DEMO_DATA | Open Data

Columns | Indexes | Properties | Runtime Information

	Name	SQL Data Type	Column Store Data Type	Key	Not Null	Default	Comment
1	ID	INTEGER	INT	1	X	NULL	
2	NAME	VARCHAR(10)	STRING			NULL	
3	DESCRIPTION	VARCHAR(100)	STRING			NULL	
4	LOCATION	VARCHAR(50)	STRING			NULL	

Eine Anpassung der Spalten ist ebenfalls möglich:

Analyze

```
1
2 Alter table "GBI_DEMO_DATA"."TEST_TABLE1" Alter ("LOCATION"VARCHAR(55));
3
4
```

Table Name: TEST_TABLE1 | Schema: GBI_DEMO_DATA | Open Data

Columns | Indexes | Properties | Runtime Information

	Name	SQL Data Type	Column Store Data Type	Key	Not Null	Default	Comment
1	ID	INTEGER	INT	1	X	NULL	
2	NAME	VARCHAR(10)	STRING			NULL	
3	DESCRIPTION	VARCHAR(100)	STRING			NULL	
4	LOCATION	VARCHAR(55)	STRING			NULL	

Problematisch ist jedoch eine Verkürzung der Länge des entsprechenden Wertes:

Analyze

```
1
2 Alter table "GBI_DEMO_DATA"."TEST_TABLE1" Alter ("LOCATION"VARCHAR(50));
3
```

SQL Console

Could not execute 'Alter table "GBI_DEMO_DATA"."TEST_TABLE1" Alter ("LOCATION"VARCHAR(50))'
Error: (dberror) [7]: feature not supported: cannot shorten field length: LOCATION: line 2 col 49 (at pos 49)

Copy | OK

Einsatz von Join-Operationen mit Hilfe SQL in SAP HANA

Anlegen der notwendigen Tabellen

Zunächst ist es notwendig einige Vorbereitungen durchzuführen. Innerhalb des vorbereiteten Datenbankschemas werden zwei Tabellen benötigt, die zu einem späteren Zeitpunkt mit Hilfe der verschiedenen Join-Operationen ausgewertet werden.

Bitte beachten Sie, da wir für die Operationen alle auf dem gleichen Datenbankschema arbeiten, müssen die Namen der Artefakte anhand folgender Regel angepasst werden.

Benutzer: D02_LEARN_xxx → D02_LEARN_000

Tabellen:
EMPLOYEE_TABLE → D000_EMPLOYEE_TABLE und D000_BOOKING_DETAILS

Die beiden Tabellen „EMPLOYEE_TABLE“ und „BOOKING_DETAILS“ sind nachfolgend grob skizziert.

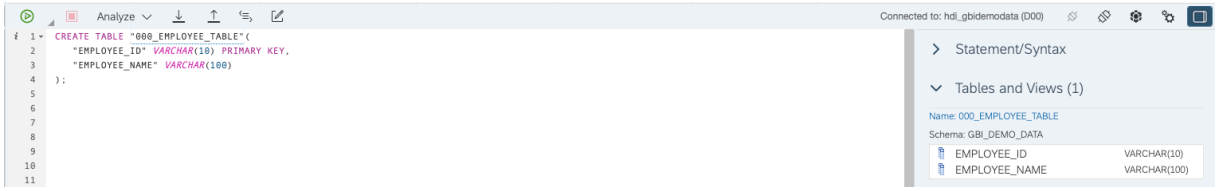
Employee Table

Employee_ID	Employee_Name
E_1	Alex
E_2	John
E_3	Marc
E_4	Peter

Booking Details

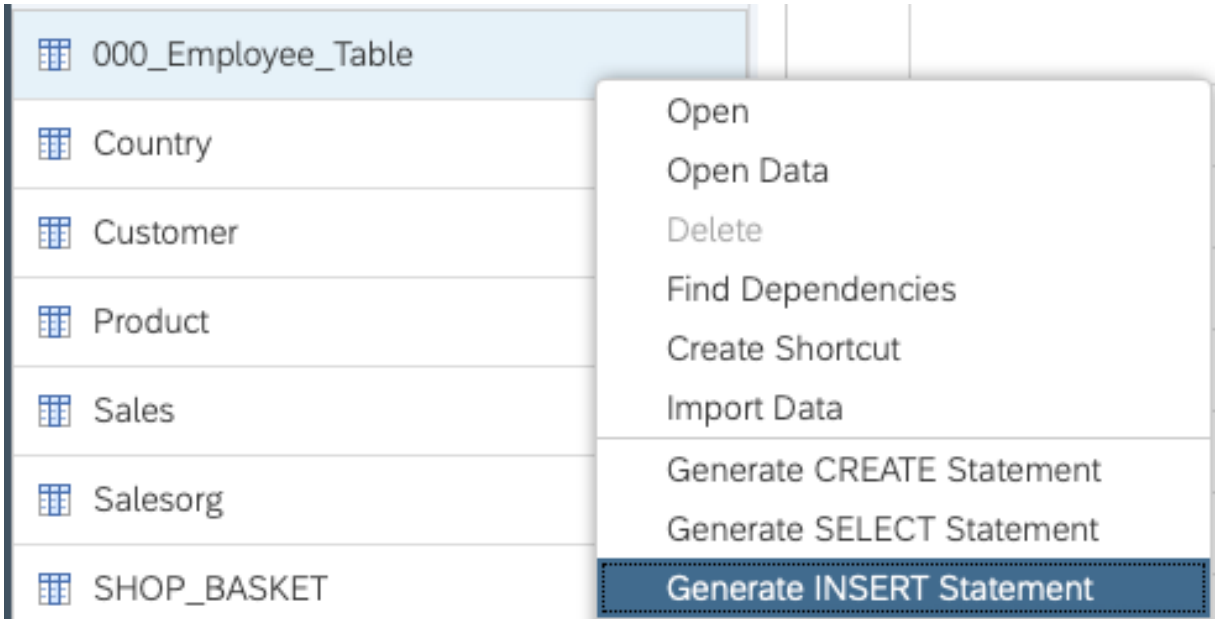
Booking_ID	Employee_ID	Product_ID	Total_Units
101	E_1	Ipod	1
102	E_1	Camera	2
103	E_3	Laptop	3
104	E_4	Mobile	4
105	E_5	Desktop	5

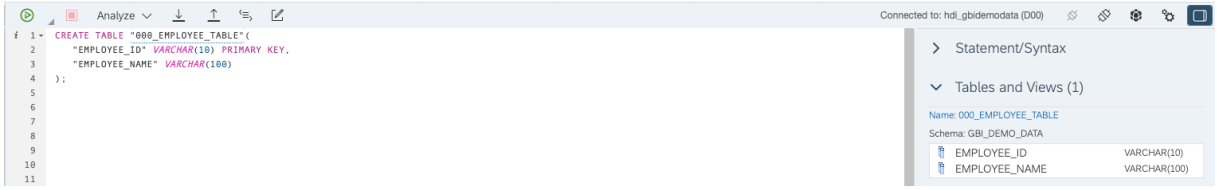
Sie werden mit Hilfe von SQL-Anweisungen angelegt.



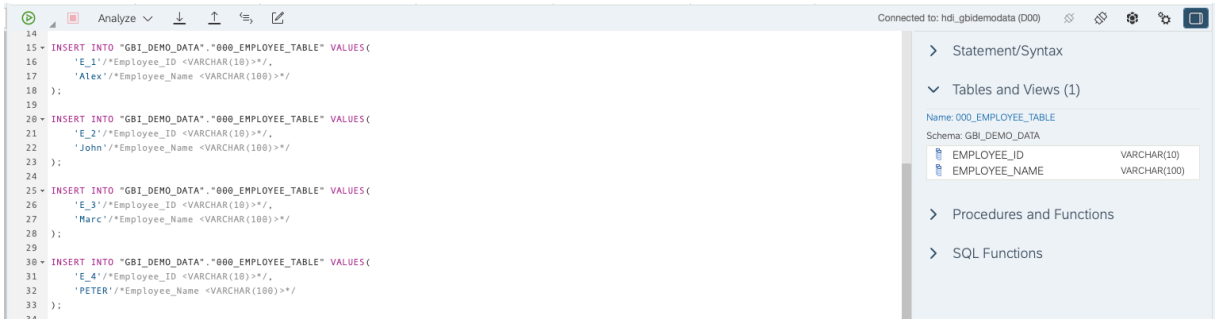
Nach erfolgreicher Anlage der Tabelle im Schema, werden 4 Datenzeilen in die Tabelle eingetragen. Das geschieht ebenfalls mit Hilfe von SQL-Anweisungen.

Über die Funktion der SQL Console werden die Anweisungsstrukturen angelegt.

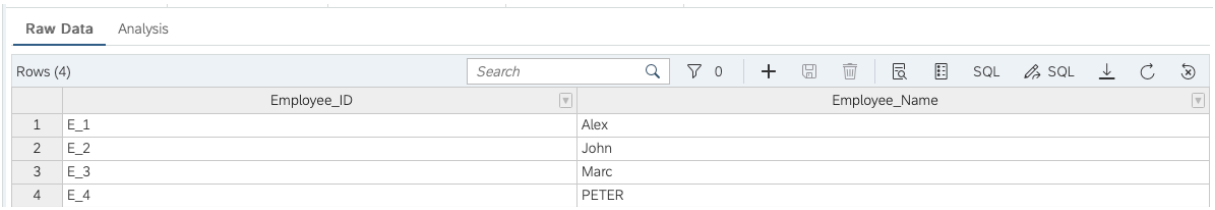




Und danach innerhalb der Console spezifiziert.



Mit Hilfe einen Rechtsklicks auf die Tabelle und der Auswahl “Open DATA“ werden die verfügbaren Einträge angezeigt.



	Employee_ID	Employee_Name
1	E_1	Alex
2	E_2	John
3	E_3	Marc
4	E_4	PETER

Nach dem gleichen Verfahren wird die 2. Tabelle angelegt und die notwendigen Einträge vorgenommen.

```
CREATE TABLE "000_BOOKING_DETAILS"(
    BOOKING_ID INTEGER PRIMARY KEY,
    EMPLOYEE_ID VARCHAR(10),
    PRODUCT_ID NVARCHAR(50),
    TOTAL_UNITS INTEGER
);

INSERT INTO "GBI_DEMO_DATA"."000_BOOKING_DETAILS" VALUES(
    101/*BOOKING_ID <INTEGER>*/,
    'E_1'/*EMPLOYEE_ID <VARCHAR(10)>*/,
    'Ipod'/*PRODUCT_ID <NVARCHAR(50)>*/,
    1/*TOTAL_UNITS <INTEGER>*/
);

INSERT INTO "GBI_DEMO_DATA"."000_BOOKING_DETAILS" VALUES(
    102/*BOOKING_ID <INTEGER>*/,
    'E_1'/*EMPLOYEE_ID <VARCHAR(10)>*/,
    'Camera'/*PRODUCT_ID <NVARCHAR(50)>*/,
    2/*TOTAL_UNITS <INTEGER>*/
);

INSERT INTO "GBI_DEMO_DATA"."000_BOOKING_DETAILS" VALUES(
    103/*BOOKING_ID <INTEGER>*/,
    'E_3'/*EMPLOYEE_ID <VARCHAR(10)>*/,
    'Laptop'/*PRODUCT_ID <NVARCHAR(50)>*/,
    3/*TOTAL_UNITS <INTEGER>*/
);

INSERT INTO "GBI_DEMO_DATA"."000_BOOKING_DETAILS" VALUES(
    104/*BOOKING_ID <INTEGER>*/,
    'E_4'/*EMPLOYEE_ID <VARCHAR(10)>*/,
    'Mobile'/*PRODUCT_ID <NVARCHAR(50)>*/,
    1/*TOTAL_UNITS <INTEGER>*/
);

INSERT INTO "GBI_DEMO_DATA"."000_BOOKING_DETAILS" VALUES(
    105/*BOOKING_ID <INTEGER>*/,
    'E_5'/*EMPLOYEE_ID <VARCHAR(10)>*/,
    'Desktop'/*PRODUCT_ID <NVARCHAR(50)>*/,
    5/*TOTAL_UNITS <INTEGER>*/
);
```

Mit Hilfe einen Rechtsklicks auf die Tabelle und der Auswahl “Open Content“, werden die verfügbaren Einträge zur Kontrolle der korrekten Ausführung angezeigt.

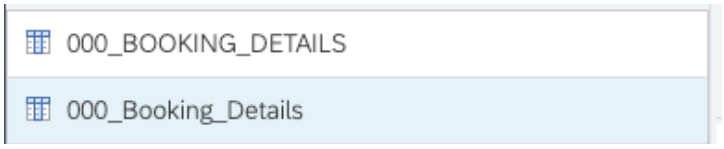
Raw Data | Analysis

Rows (5)

	BOOKING_ID	EMPLOYEE_ID	PRODUCT_ID	TOTAL_UNITS
1	101	E_1	Ipod	1
2	105	E_5	Desktop	5
3	102	E_1	Camera	2
4	103	E_3	Laptop	3
5	104	E_4	Mobile	1

Wichtig:

Bei der Definition der Tabellennamen und Tabellenspalten ist auf Groß- und Kleinschreibung zu achten, da bereits die unterschiedliche Schreibweise zu mehreren Artefakten führt.



Definition der verfügbaren Join-Operationen

Abfragen über mehrere Datenbanktabellen realisiert man im relationalen Datenbankmodell mithilfe von SQL-JOINs. Bei den verfügbaren JOIN-Typen innerhalb von SAP HANA handelt es sich um eine Kombination aus kartesischem Produkt und Selektion.

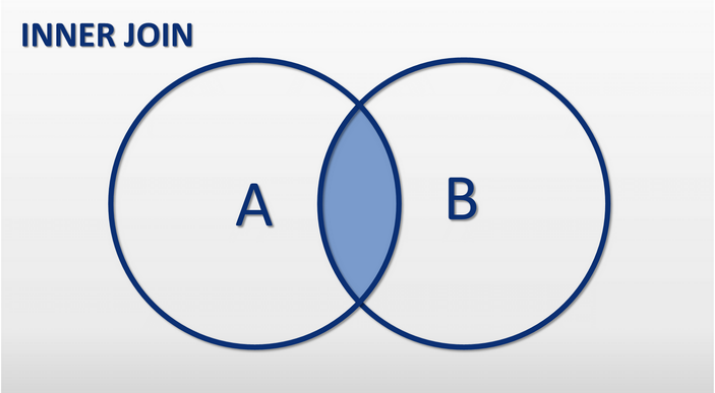
Das Datenbank-Managementsystem (DBMS) bildet zunächst das **Kreuzprodukt** zweier Datenbanktabellen. Anschließend filtert es das Ergebnis gemäß einer vom Anwender via SQL-Statement definierten **Selektionsbedingung**. Der INNER JOIN hebt sich dabei von allen anderen JOIN-Typen durch eine **minimale Ergebnismenge** ab. Als Ergebnis eines INNER JOINs werden lediglich die Datensätze des Kreuzproduktes ausgegeben, die die Selektionsbedingung erfüllen. Das Resultat ist eine Ergebnistabelle (ein *View*) ohne Nullwerte.

Innerhalb von SAP HANA stehen verschiedene Join-Operationen zu Verfügung und werden nachfolgend einzeln dargestellt.

- Inner Join
- Left Outer Join
- Right Outer Join
- Full Outer Join

Inner Join -

Mit Hilfe der “Inner Join Operation” werden alle Einträge ausgewählt, die in beiden Ausgangstabellen beinhaltet sind und ergeben damit die Schnittmenge beider Tabellen.



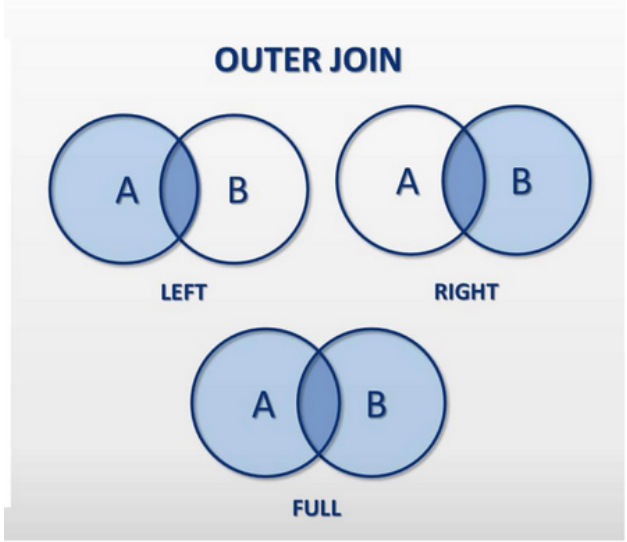
SQL Code

```
-- Inner Join selects the set of records that match in both the tables
SELECT A2."BOOKING_ID", A1."EMPLOYEE_ID", A1."EMPLOYEE_NAME", A2."PRODUCT_ID",
A2."TOTAL_UNITS"
FROM "000_EMPLOYEE_TABLE" AS A1
INNER JOIN "000_BOOKING_DETAILS" AS A2
ON A1."EMPLOYEE_ID" = A2."EMPLOYEE_ID";
```

SQL Outer Joins

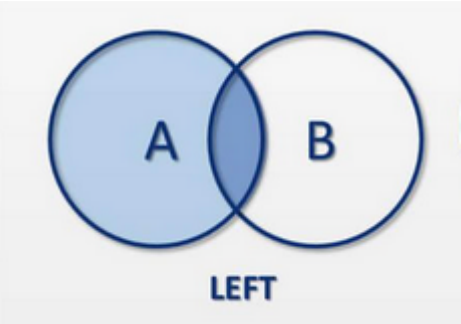
Ein **SQL OUTER JOIN** hingegen gibt nicht nur die Datensätze beider Tabellen aus, die die Selektionsbedingung (beispielsweise die Gleichheit der Werte zweier Spalten) erfüllen, sondern zusätzlich auch **alle übrigen Tupel der einen bzw. der anderen Tabelle**.

Bezogen auf die Leserichtung der SQL-Syntax spricht man von einer linken und einer rechten Tabelle. Die jeweiligen Operationen heißen dementsprechend **LEFT OUTER JOIN** und **RIGHT OUTER JOIN**. Möchten Sie im Rahmen von Datenbankabfragen zusätzlich zu den Datensätzen, die die Selektionsbedingung erfüllen, auch sämtliche Datensätze der linken und rechten Tabelle ausgeben, handelt es sich um einen **FULL OUTER JOIN**.



Left Outer Join -

Der Left Outer Join selektiert die kompletten Datensätze der “ersten” Tabelle, welche zu den Einträgen in der “zweiten / rechten” Tabelle passen. Sollten in der zweiten Tabelle keine passenden Einträge vorhanden sein, dann enthält die rechte Seite der Ergebnismenge den Wert „null“.



SQL Code

```
SELECT A2."BOOKING_ID", A1."EMPLOYEE_ID", A1."EMPLOYEE_NAME", A2."PRODUCT_ID",
A2."TOTAL_UNITS"
FROM "000_EMPLOYEE_TABLE" AS A1
LEFT OUTER JOIN "000_BOOKING_DETAILS" AS A2
ON A1."EMPLOYEE_ID" = A2."EMPLOYEE_ID";
```

Result × Messages × History

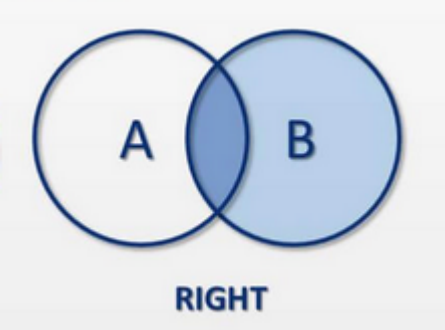
Rows (5) SQL

```
SELECT A2."BOOKING_ID", A1."EMPLOYEE_ID", A1."EMPLOYEE_NAME", A2."PRODUCT_ID", A2."TOTAL_UNITS"
FROM "000_EMPLOYEE_TABLE" AS A1
LEFT OUTER JOIN "000_BOOKING_DETAILS" AS A2
ON A1."EMPLOYEE_ID" = A2."EMPLOYEE_ID"
```

	BOOKING_ID	EMPLOYEE_ID	EMPLOYEE_NAME	PRODUCT_ID	TOTAL_UNITS
1	101	E_1	Alex	Ipod	1
2	102	E_1	Alex	Camera	2
3	103	E_3	Marc	Laptop	3
4	104	E_4	PETER	Mobile	1
5	*NULL*	E_2	John	*NULL*	*NULL*

Right Outer Join -

Der Right Outer Join selektiert die kompletten Datensätze der “zweiten/rechten” Tabelle, welche zu den Einträgen in der „ersten/linken” Tabelle passen. Sollten in der ersten Tabelle keine passenden Einträge vorhanden sein, dann enthält die linke Seite der Ergebnistabelle den Wert „null“.



SQL Code

```
SELECT A2."BOOKING_ID", A1."EMPLOYEE_ID", A1."EMPLOYEE_NAME", A2."PRODUCT_ID",
A2."TOTAL_UNITS"
FROM "000_EMPLOYEE_TABLE" AS A1
RIGHT OUTER JOIN "000_BOOKING_DETAILS" AS A2
ON A1."EMPLOYEE_ID" = A2."EMPLOYEE_ID";
```

Result × Messages × History

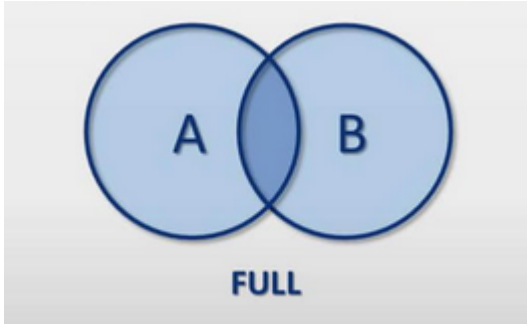
Rows (5) SQL

```
SELECT A2."BOOKING_ID", A1."EMPLOYEE_ID", A1."EMPLOYEE_NAME", A2."PRODUCT_ID", A2."TOTAL_UNITS"
FROM "000_EMPLOYEE_TABLE" AS A1
RIGHT OUTER JOIN "000_BOOKING_DETAILS" AS A2
ON A1."EMPLOYEE_ID" = A2."EMPLOYEE_ID"
```

	BOOKING_ID	EMPLOYEE_ID	EMPLOYEE_NAME	PRODUCT_ID	TOTAL_UNITS
1	101	E_1	Alex	Ipod	1
2	102	E_1	Alex	Camera	2
3	103	E_3	Marc	Laptop	3
4	104	E_4	PETER	Mobile	1
5	105	*NULL*	*NULL*	Desktop	5

Full Outer Join -

Bei einem **FULL OUTER JOIN** handelt es sich um eine Kombination aus LEFT OUTER JOIN und RIGHT OUTER JOIN. Der FULL JOIN verbindet die Datensätze der Ausgangstabellen gemäß der Selektionsbedingung, führt jedoch nicht nur die verbundenen Datensätze in der Ergebnistabelle auf, sondern auch die Datensätze beider Tabellen, die die Selektionsbedingung nicht erfüllen. Auch beim FULL JOIN werden fehlende Werte auf NULL gesetzt.



SQL Code

```
SELECT A2."BOOKING_ID", A1."EMPLOYEE_ID", A1."EMPLOYEE_NAME", A2."PRODUCT_ID",
A2."TOTAL_UNITS"
FROM "000_EMPLOYEE_TABLE" AS A1
FULL OUTER JOIN "000_BOOKING_DETAILS" AS A2
ON A1."EMPLOYEE_ID" = A2."EMPLOYEE_ID";
```

Result × Messages × History

Rows (6) SQL

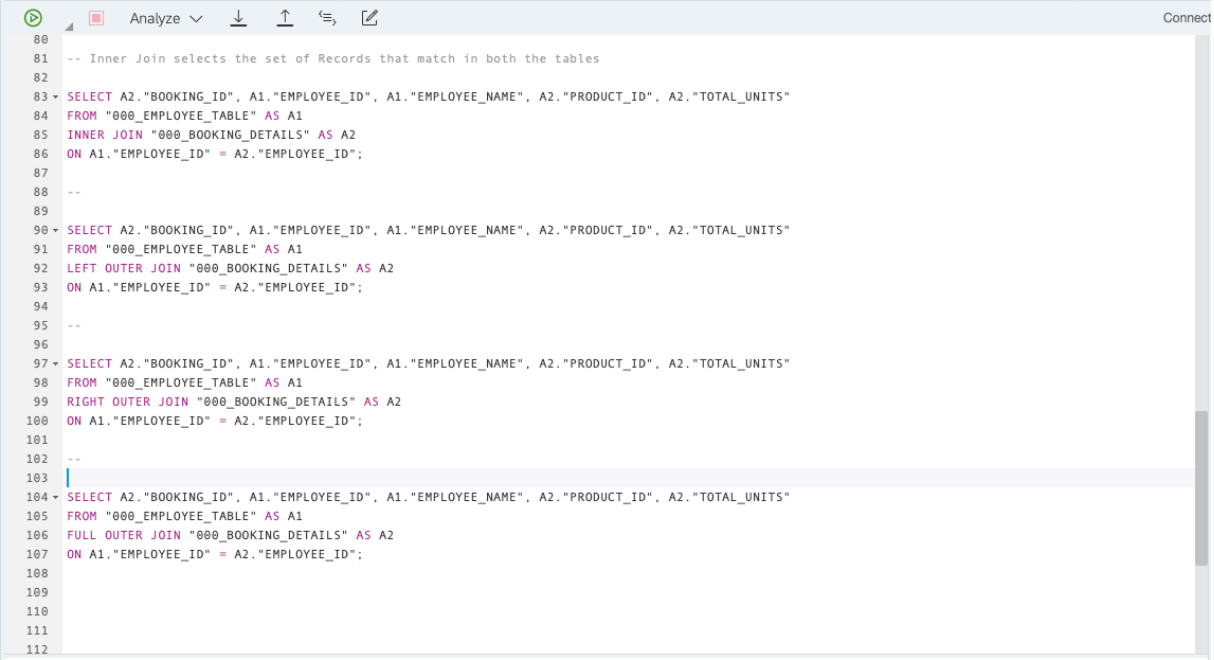
```
SELECT A2."BOOKING_ID", A1."EMPLOYEE_ID", A1."EMPLOYEE_NAME", A2."PRODUCT_ID", A2."TOTAL_UNITS"
FROM "000_EMPLOYEE_TABLE" AS A1
FULL OUTER JOIN "000_BOOKING_DETAILS" AS A2
ON A1."EMPLOYEE_ID" = A2."EMPLOYEE_ID"
```

	BOOKING_ID	EMPLOYEE_ID	EMPLOYEE_NAME	PRODUCT_ID	TOTAL_UNITS
1	101	E_1	Alex	Ipod	1
2	102	E_1	Alex	Camera	2
3	103	E_3	Marc	Laptop	3
4	104	E_4	PETER	Mobile	1
5	105	*NULL*	*NULL*	Desktop	5
6	*NULL*	E_2	John	*NULL*	*NULL*

Persistieren von Anfragen

Innerhalb der SAP Web IDE wird die SQL Console als Werkzeug zur direkten Eingabe von Statements verwendet.

Hierin ist es möglich und häufig sinnvoll mehrere Statements zu definieren und über die Speicherfunktion diese auch in einer Datei zu speichern.



Import der Anweisungen aus einer lokalen Datei

Import bereits abgespeicherter Anweisungen. Innerhalb der SQL-Console kann mit Hilfe des Kontextmenüs via „Open File“ eine lokale Datei importiert werden.

Die betreffende Datei wird im Auswahldialog selektiert und geöffnet. Der Inhalt wird innerhalb der Console angezeigt und kann hierin ausgeführt und editiert werden.

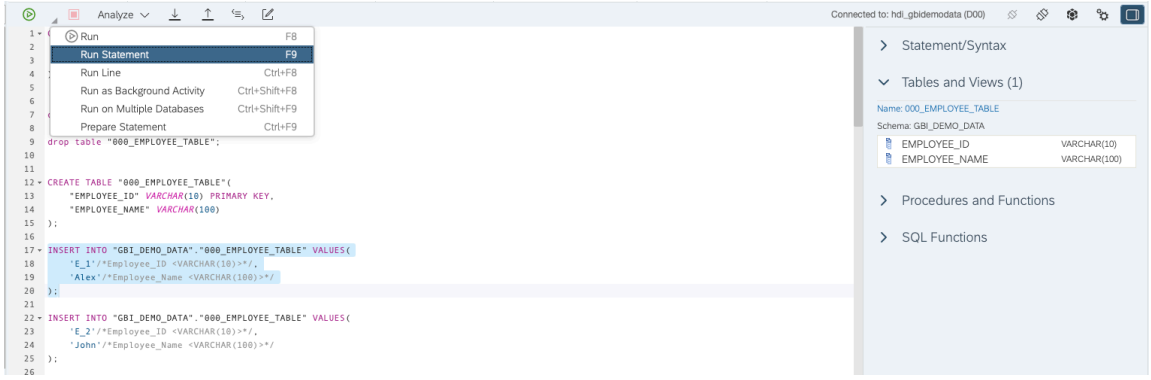
Export der Anweisungen in eine lokale Datei

Vergleichbar mit der Importfunktionalität kann auch das Abspeichern des Consolen-Inhaltes in eine lokale Datei genutzt werden.

Über das Kontextmenü der Console wird die Funktion „Save as“ ausgewählt und der komplette Inhalt in eine „.sql“-Datei abgespeichert. Die gleiche Funktion kann auch über die „Save as“ Funktion des Menüs ausgewählt werden.

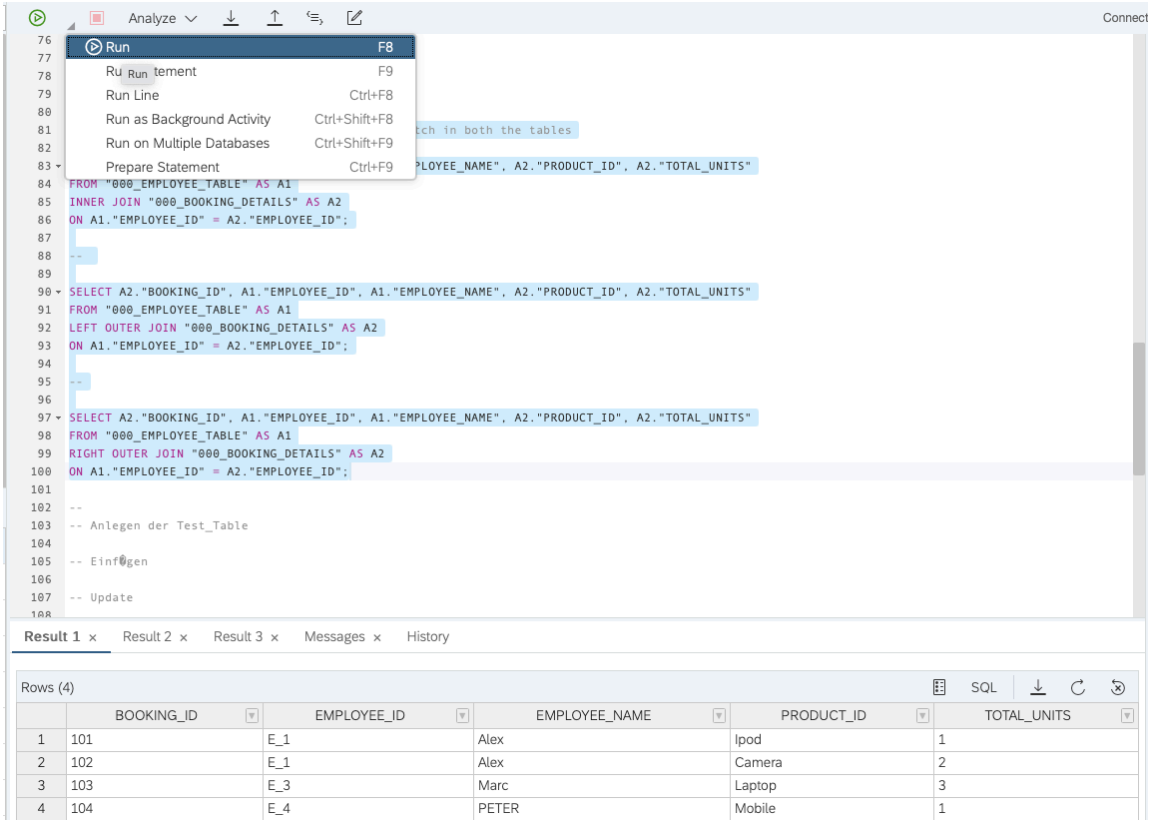
Selektives Ausführen einzelner Anfragen

Wenn nur einzelne Anweisungen ausgeführt werden sollen, müssen diese explizit markiert werden, bevor die Ausführung angestoßen wird. Hierzu markiert man die entsprechenden Anweisungsblöcke.

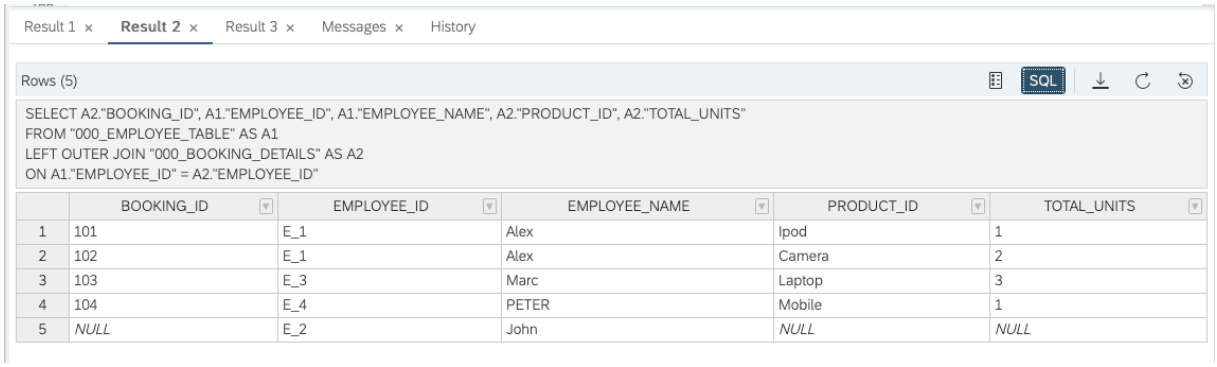


Zur Ausführung der Anweisungen wird dann wieder das „Run“ Icon .

Es ist ebenfalls möglich mehrere Anweisungen auf einmal zu prozessieren. Hierzu werden entweder die benötigten Anweisungen selektiert oder aber keine Selektion vorgenommen.



Jedes Ergebnis einer Abfrage wird danach in einem separaten Tab dargestellt. Die zu Grunde liegende Anweisung ist im Kopf des Bildschirms dargestellt.



Einfügen von Kommentaren zur Erläuterung innerhalb der Anweisungen

Um Anweisungen mit Hilfe einer Inline-Dokumentation zu versehen oder Kommentierungen vorzunehmen, wird innerhalb der Blöcke das doppelte Minuszeichen eingesetzt.